

Jason Ablott

11/25/2024

CS-6012

HW 7 Analysis: Hash Tables

1. BadHashFuncor:

This classes hash method returns the ascii value of the first character in the string item minus the ascii value of 'A'. Given that a string starts with a letter and not a special character or space, we expect this function to only be able to realistically return around 52 unique hash values. This will cause very poor performance in most cases, especially as set sizes get larger than 52 as collisions are a certainty at that point.

2. MediocreHashFuncor:

This classes hash method returns 31 (a prime number) plus the sum of the ascii values of a string, all divided by the sum of the first and last items' ascii values multiplied by 31 (the same prime number as started with). The original version of this did not include the prime numbers as the starting value and multiplier of the denominator, and the performance was quite poor. To make the method a bit better the prime numbers were added to generate a larger and more unique range of possible results. By including the addition of the prime numbers this increases the performance substantially (maybe more than intended), so the method is almost comparable to the good hash functor.

3. GoodHashFuncor:

This classes hash method uses a variation of the djb2 algorithm written by Dan Bernstein. This method starts with a unique prime number (5381) and then traverses the characters of the string, continuing to add to the hash value with each character. At each character, the hash has bitwise operations applied that effectively multiply the value by 33, then the ascii value of the character is added. This is repeated for every character. Finally, the absolute value of the hash is returned to ensure all positive values are returned. This modification reduces the effectiveness slightly, but the algorithm maintains very good performance. This is due in large part to the start at a unique prime number, and then the multiplication that takes place at each character spreading out the output hash values over the possible range of integer values.

4. Assessment of Quality and Efficiency of Each Hash Function.

To assess each of the three hash functions, a total of 11 experiments were performed. First, each hash function is used to add varying amounts of strings read in from the book

Moby Dick into a hash table. Sizes varied from 10 to 81920 strings added to hash tables of equivalent capacity. One table using each hash function was tested to compare the number of collisions (items that were placed at the same index in the hash table) and the lambda value of the table (the length of the longest list at any given index i.e. most items in a bucket). This experiment demonstrates the poor performance of the bad hash method, the much better performance of the mediocre hash method, and the best performance of the good hash method. As a quick comparison, after adding 81920 strings to the table, the bad hash resulted in 10410 collisions and a lambda of 1298, vs the good hash with 686 collisions and a lambda of 5. See figures 1 and 2 below for the complete results of these first 2 experiments.

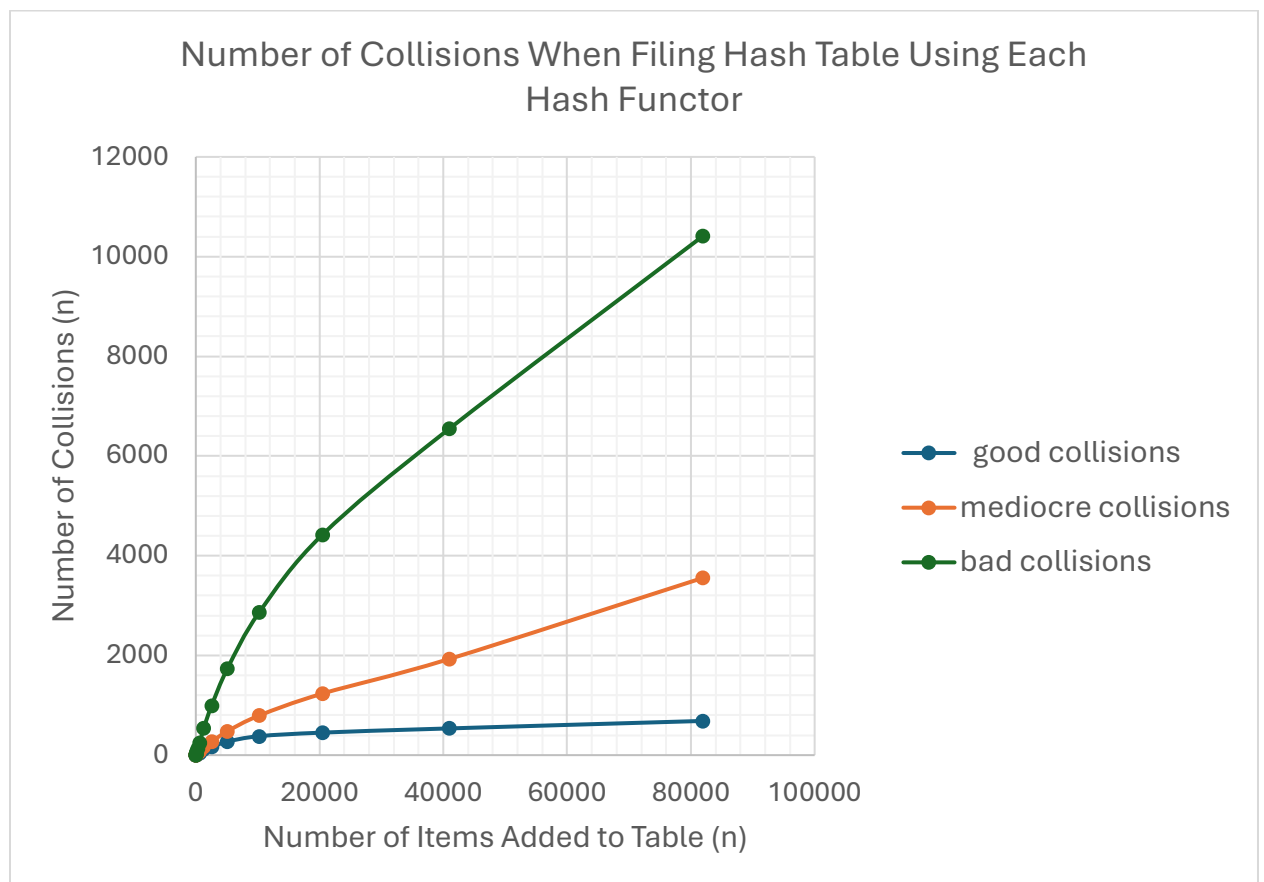


Figure 1. How hash functions affect collisions in resulting hash table creation.

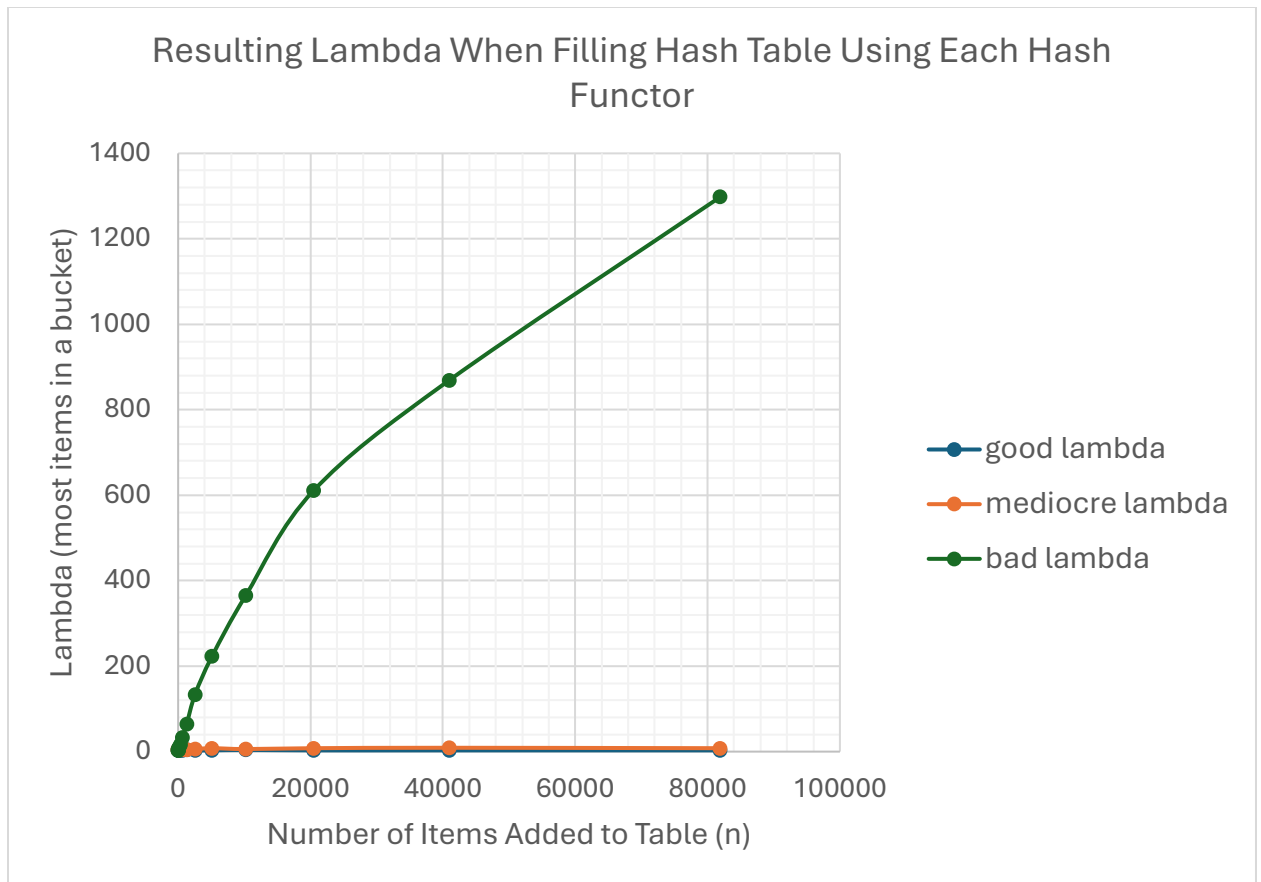


Figure 2. How hash functions affect lambda values in resulting hash tables.

Next, 9 more experiments were conducted to assess the variance of time it takes to use the `add()`, `remove()`, and `contains()` methods. Each method was timed on a hash table that used each of the 3 hash functions. The results of the tests for each method were compared to each-other for each hash function to determine the effect of the hash function on the method.

First, the `add` method was compared. All results were very comparable for the `add` method. At first this may seem unexpected, but upon inspection of the `LinkedList` class that is used for each index in the hash table, it does make sense. Since the `LinkedList` class is a doubly linked list, it allows $O(1)$ adds to the end of the linked list, which means that it is actually faster to add an item to a linked list than to an empty index (where a new linked list has to be initialized). The bad hash function is also slightly faster than the mediocre and good functions as it only uses the first character in the item to generate the hash code, where the other two use every character. Because of these reasons, the bad hash table performs slightly better as many items ended up in the same indices in the table which saved time when adding to the table, as well as the time to generate the hash codes being slightly faster. See figure 3 below for the results of the `add()` tests.

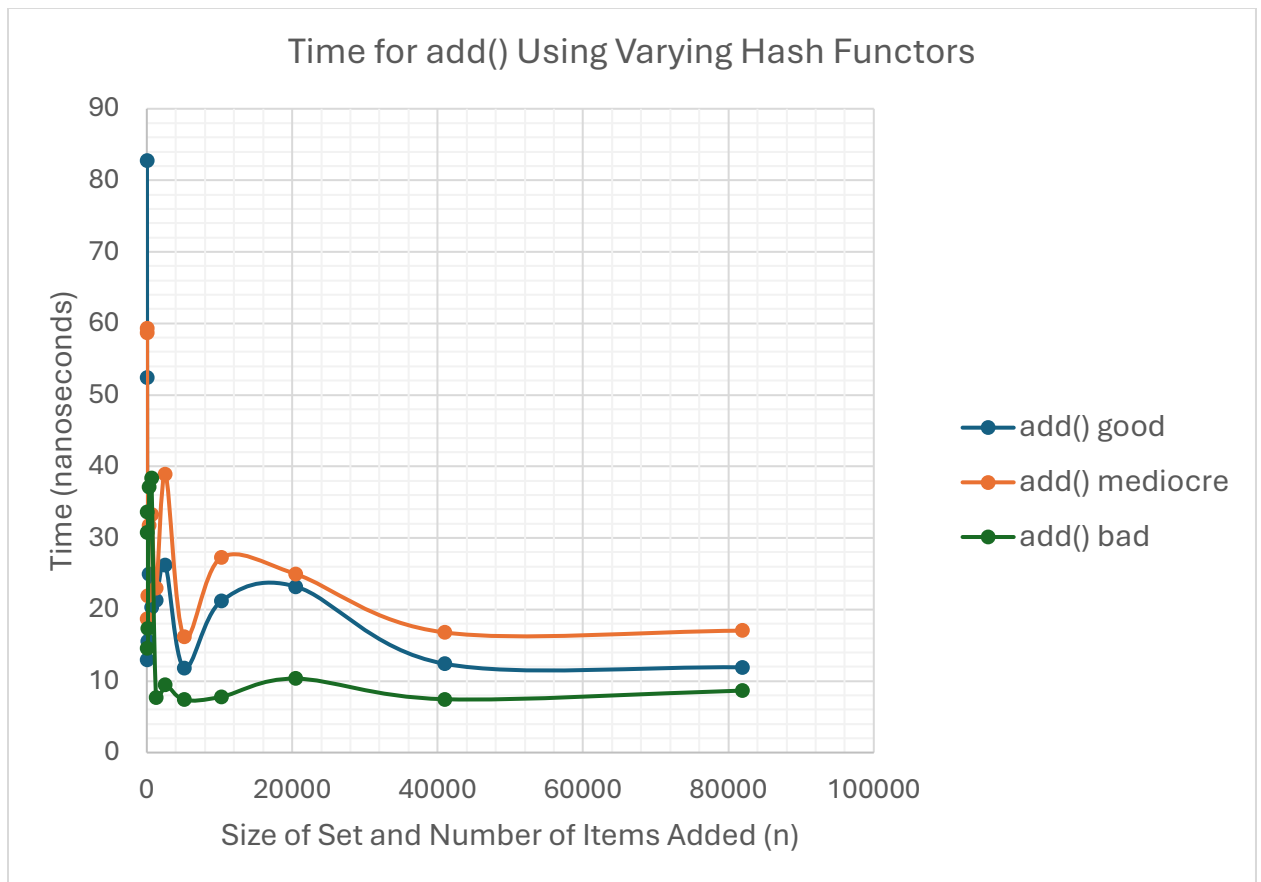


Figure 3. How different hash methods affect adding a single element.

Second, the `contains()` method was tested with hash tables using each of the three hash functions. The results are as expected, though the distribution of the data is perhaps a bit strange. Looking at the results (see figure 4 below), we can see that the good hash table slightly outperforms the mediocre hash table, but the bad table is much slower, though only in certain parts of the table. This is because the bad hash table puts most of its items into a cluster due to the fact that it generally only returns around 52 unique hash values. Because of this, areas around the main cluster experience much slower times to find elements because they must traverse long linked lists to find the target. Outside of the cluster, most indices are empty or contain very few items, thus search times are fast. For the good and mediocre tables, the results are consistent throughout indicating an even distribution of items.

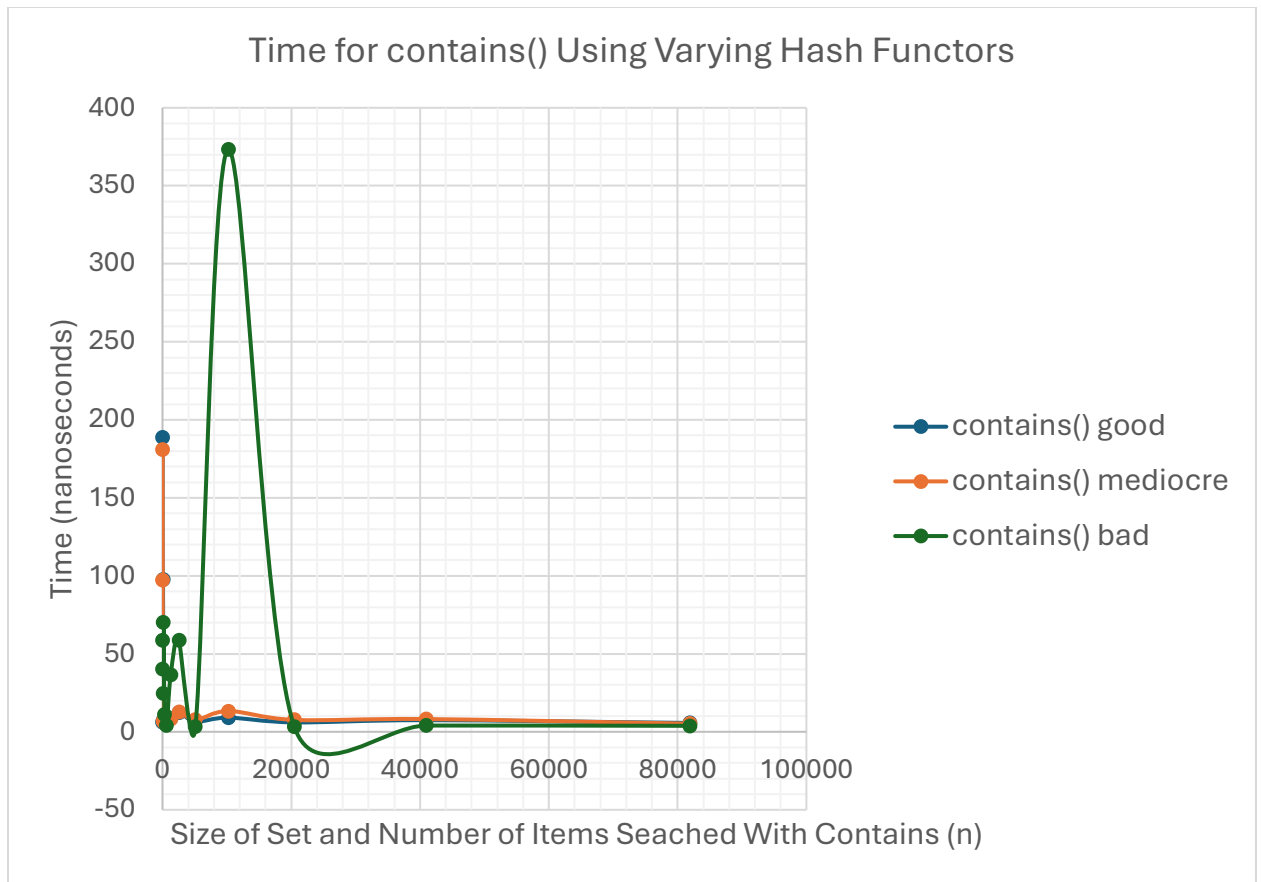


Figure 4. How different hash methods affect time to find items in a hash table

Finally, the `remove()` method was tested on each type of table. The results are as expected, with the good and mediocre tables both performing consistently and well to remove items (good does slightly outperform mediocre, though both are within a range of around 20 nanoseconds or less of each other). The bad table takes significantly longer (up to 50x) to remove items, because they are so unevenly distributed and remove has to traverse long linked lists a large percentage of the time to find the elements to remove. See figure 5 below for the results of this test.

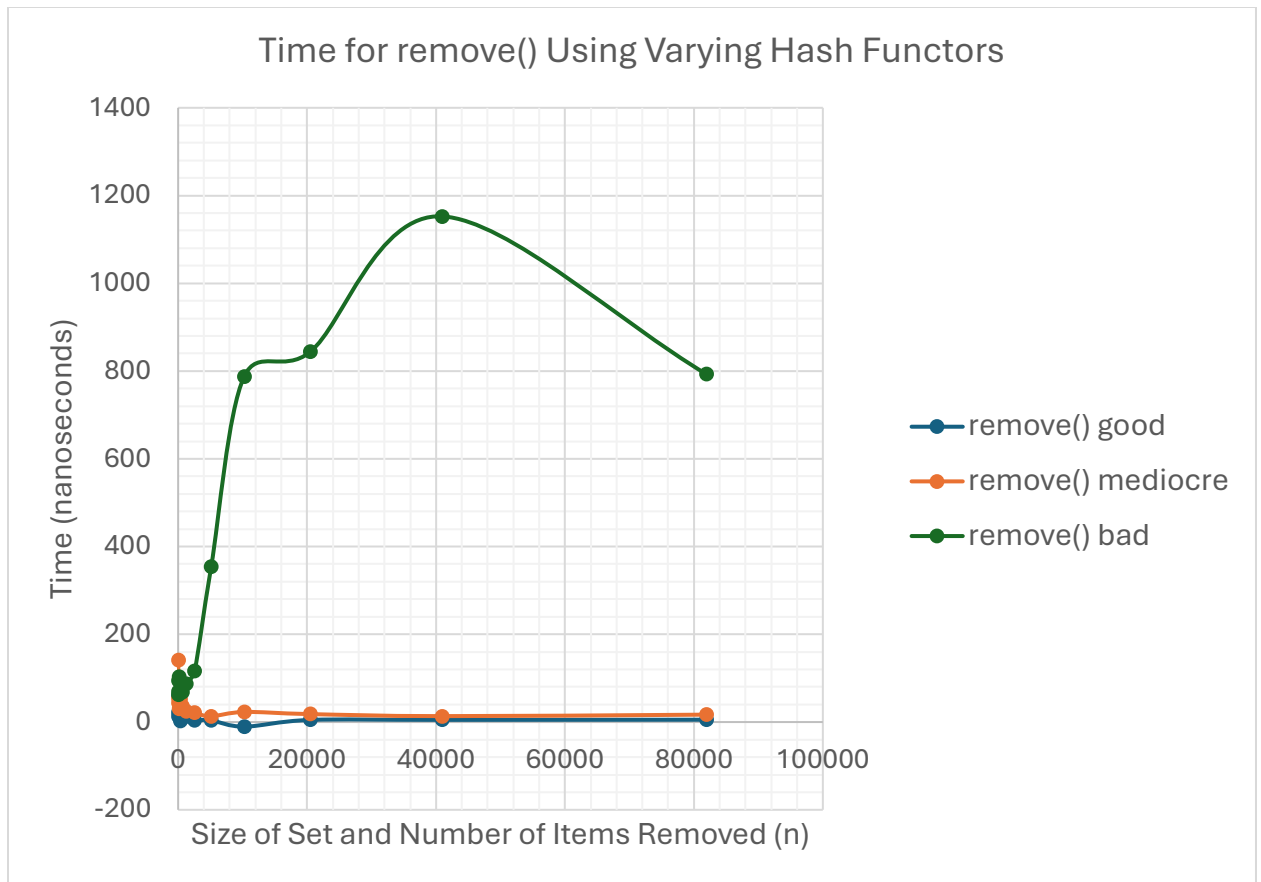


Figure 5. How different hash functions affect speed of removing elements.

5. Hash Function Performance:

The hash functions performed as expected and as discussed previously in parts 1-3. The results prove the performance of each hash function and demonstrate the importance of choosing a good hash function when creating a hash table. The performance of the hash functions is as follows:

- BadHashFuncor.hash(item) has a $O(1)$ as it only uses the 1st element in the item
- MediocreHashFuncor.hash(item) has a $O(N)$ performance as it uses every character in item.
- GoodHashFuncor.hash(item) has a $O(N)$ performance as it uses every character in item.